

**UNIVERSIDADE DE MOGI DAS CRUZES
CURSO DE SISTEMAS DE INFORMAÇÕES**

BRUNO BERNARDELLI FRAUDIO

**LOGIN SEGURO: SISTEMA DE AUTENTICAÇÃO E AUTORIZAÇÃO
COM SPRING BOOT, THYMELEAF E MONGODB ATLAS**

Documentação técnica do sistema

Mogi das Cruzes
2026

SUMÁRIO

1 INTRODUÇÃO	3
2 TECNOLOGIAS UTILIZADAS	3
3 ESTRUTURA DO SISTEMA	4
3.1 Organização em camadas	4
3.2 Camada de modelo	4
3.3 Camada de repositório	5
3.4 Camada de serviço	5
3.5 Camada de apresentação	6
4 INTEGRAÇÃO COM O MONGODB ATLAS	6
4.1 Configuração da conexão	6
4.2 Preparação do cluster	7
4.3 Armazenamento das sessões	7
5 CONTROLE DE ACESSO	8
5.1 Perfis de usuário	8
5.2 Proteção das rotas	8
5.3 Criptografia das senhas	9
5.4 Medidas complementares	9
6 PRINCIPAIS DECISÕES DE DESIGN	10
6.1 Separação entre tema e estrutura	10
6.2 Perfis como enum	10
6.3 Separação entre formulário e entidade	11
6.4 Criação automática do administrador	11
6.5 Sessões gravadas no banco	11
6.6 Síntese das decisões	12
7 GUIA PARA FUTURAS ADAPTAÇÕES	12
7.1 Acrescentar um novo perfil	12
7.2 Criar um tema visual	13
7.3 Adicionar campos ao usuário	13
7.4 Criar uma nova página protegida	13
7.5 Substituir o banco de dados	13
8 VERSIONAMENTO COM GITFLOW	14
9 CONSIDERAÇÕES FINAIS	14

REFERÊNCIAS..... 16

1 INTRODUÇÃO

Este documento explica a estrutura e o funcionamento do sistema Login Seguro, uma aplicação web criada para controlar o acesso de usuários. O sistema Login Seguro permite o cadastro de contas, a autenticação com e-mail e senha, e a liberação de páginas conforme o perfil de cada usuário.

A aplicação Login Seguro foi construída com Java 17 e Spring Boot 3. Ela usa o Spring Security para autenticação e autorização, o Thymeleaf para gerar as páginas e o MongoDB Atlas como banco de dados em nuvem. As senhas são guardadas apenas como hash, e as credenciais de conexão ficam fora do código-fonte.

O objetivo deste texto é registrar como o sistema Login Seguro foi organizado e explicar por que cada decisão foi tomada, para que outra pessoa consiga dar manutenção ou adaptar o projeto sem precisar reconstruir o raciocínio do início. A seção 7 reúne roteiros práticos para as adaptações mais prováveis.

2 TECNOLOGIAS UTILIZADAS

O quadro a seguir relaciona as tecnologias empregadas no projeto e a função de cada uma na aplicação.

Quadro 1 – Tecnologias utilizadas no projeto

Tecnologia	Versão	Função no projeto
Java	17	Linguagem de programação da aplicação
Spring Boot	3.2.5	Estrutura base, injeção de dependências e servidor embutido
Spring Security	6.2	Autenticação, autorização e proteção contra CSRF
Spring Data MongoDB	4.2	Mapeamento de objetos Java para documentos do banco
Spring Session	3.2	Armazenamento das sessões no banco de dados
Thymeleaf	3.1	Geração das páginas HTML no servidor
MongoDB Atlas	8.0	Banco de dados em nuvem
Maven	3.9	Gerenciamento de dependências e construção do projeto

Fonte: elaborado pelo autor (2026).

O MongoDB foi escolhido por ser um banco de dados orientado a documentos, o que facilita o armazenamento da lista de perfis diretamente no registro do usuário, sem a necessidade de uma tabela de associação, como ocorreria em um banco de dados relacional.

3 ESTRUTURA DO SISTEMA

3.1 Organização em camadas

O projeto está organizado em pacotes, cada um com uma responsabilidade definida. Essa separação segue o padrão em camadas, no qual cada camada conhece apenas a camada imediatamente inferior. O objetivo é permitir que uma parte seja alterada sem afetar as demais.

Quadro 2 – Pacotes do projeto e suas responsabilidades

Pacote	Responsabilidade
model	Entidades gravadas no banco e o enum de perfis
repository	Interface de acesso ao MongoDB
service	Regras de negócio do cadastro e da gestão de usuários
controller	Recebimento das requisições e escolha da página de resposta
security	Adaptação da entidade Usuario para o Spring Security
config	Configuração de segurança, tema e carga inicial
dto	Objetos que transportam os dados dos formulários

Fonte: elaborado pelo autor (2026).

O fluxo de uma requisição percorre as camadas na seguinte ordem: o navegador envia a requisição ao controller, que chama o service; o service aplica as regras de negócio e usa o repository para gravar ou ler dados no MongoDB; o resultado retorna ao controller, que escolhe o template Thymeleaf a ser exibido.

3.2 Camada de modelo

A classe Usuario representa a conta de uma pessoa no sistema. Ela é anotada com @Document, o que indica que seus objetos são armazenados na coleção usuarios do MongoDB. O campo email recebe a anotação @Indexed com a

propriedade unique, o que faz com que o próprio banco rejeite o cadastro de dois usuários com o mesmo endereço de email.

```
@Document(collection = "usuarios")
public class Usuario {
    @Id
    private String id;
    private String nome;
    @Indexed(unique = true)
    private String email;
    private String senha;
    private List<Perfil> perfis = new ArrayList<>();
    private boolean ativo = true;
    private LocalDateTime criadoEm = LocalDateTime.now();
}
```

O campo de senha nunca recebe a senha digitada pelo usuário. O que se grava ali é o hash gerado pelo BCrypt, conforme tratado na seção 5.3. O campo ativo permite desativar uma conta sem apagá-la, preservando o histórico de quem já utilizou o sistema.

Os perfis são representados pelo enum Perfil, que define as três funções previstas no sistema e já monta o nome da autoridade no formato esperado pelo Spring Security.

```
public enum Perfil {
    ADMIN("Administrador"),
    GERENTE("Gerente"),
    USUARIO("Usuario");

    public String getAuthority() {
        return "ROLE_" + name();
    }
}
```

3.3 Camada de repositório

A interface `UsuarioRepository` estende `MongoRepository` e não possui implementação manual. O Spring Data gera o código de acesso ao banco em tempo de execução, interpretando o nome de cada método declarado.

```
public interface UsuarioRepository extends MongoRepository<Usuario,
String> {
    Optional<Usuario> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

O método `findByEmail` corresponde a uma consulta que filtra a coleção pelo campo email. Para criar novas consultas, basta declarar métodos seguindo a mesma convenção de nomes, como `findByAtivoTrue` ou `findByNomeContaining`.

3.4 Camada de serviço

A classe `UsuarioService` concentra as regras de negócio. É nela que se verifica a duplicidade de e-mail, se aplica o hash na senha e se define o perfil inicial de quem se cadastra. Os controllers não conhecem o MongoDB: eles conversam apenas com o service.

O método de cadastro normaliza o e-mail para letras minúsculas antes de gravar, de modo que os endereços `joao@exemplo.com` e `JOAO@Exemplo.com` sejam tratados como a mesma pessoa.

```
public Usuario cadastrar(CadastroForm form) {
    String email = form.getEmail().trim().toLowerCase();
    if (repository.existsByEmail(email)) {
        throw new EmailJaCadastradoException(email);
    }
    Usuario usuario = new Usuario();
    usuario.setNome(form.getNome().trim());
    usuario.setEmail(email);
    usuario.setSenha(passwordEncoder.encode(form.getSenha()));
    usuario.setPerfis(List.of(Perfil.USUARIO));
    return repository.save(usuario);
}
```

3.5 Camada de apresentação

As páginas são escritas em Thymeleaf e ficam no diretório `src/main/resources/templates`. O arquivo `fragmentos/layout.html` reúne as partes repetidas em todas as telas, como o cabeçalho com o menu e o rodapé. As demais páginas apenas referenciam esses fragmentos.

```
<head th:replace="~{fragmentos/layout ::
cabecalho('Painel')}"></head>
<header th:replace="~{fragmentos/layout :: menu}"></header>
<footer th:replace="~{fragmentos/layout :: rodape}"></footer>
```

Com essa organização, uma alteração no menu precisa ser feita em um único arquivo e passa a valer em todas as telas do sistema.

4 INTEGRAÇÃO COM O MONGODB ATLAS

4.1 Configuração da conexão

O MongoDB Atlas é o serviço de banco de dados em nuvem mantido pela própria MongoDB. A aplicação se conecta a ele por meio de uma string no formato `mongodb+srv`, que o driver utiliza para localizar os servidores do cluster a partir de registros DNS.

A conexão é declarada no arquivo `application.properties`. As credenciais não aparecem escritas ali: o que há são referências a variáveis de ambiente, que são resolvidas quando a aplicação é iniciada.

```
spring.data.mongodb.uri=${MONGODB_URI}
spring.data.mongodb.database=${MONGODB_DATABASE:login_seguro}
spring.data.mongodb.auto-index-creation=true
```

A notação `${MONGODB_DATABASE:login_seguro}` indica que, caso a variável não exista, o valor `login_seguro` será utilizado. Já a variável `MONGODB_URI` não possui valor padrão: sem ela, a aplicação não inicia, evitando que o sistema suba apontando para um banco de dados incorreto.

A propriedade `auto-index-creation` faz o Spring criar no banco os índices declarados nas entidades, entre eles o índice único sobre o campo `email`.

4.2 Preparação do cluster

Antes da primeira execução, três configurações precisam ser feitas no painel do Atlas. A primeira é a criação de um usuário do banco de dados em Database Access, com permissão de leitura e escrita. Esse usuário é distinto da conta utilizada para acessar o site do Atlas.

A segunda é a liberação do endereço IP em Network Access. O Atlas bloqueia, por padrão, qualquer conexão proveniente de endereços não cadastrados. Durante o desenvolvimento, pode-se liberar a faixa `0.0.0.0/0`, prática que não deve ser mantida em ambiente de produção.

A terceira é a obtenção da string de conexão, disponível em Clusters, opção Connect, item Drivers. Caracteres especiais presentes na senha precisam ser codificados: o caractere arroba, por exemplo, deve ser substituído por `%40`.

4.3 Armazenamento das sessões

Além dos usuários, o sistema grava as sessões ativas no próprio MongoDB, por meio da biblioteca Spring Session. Essa escolha é discutida na seção 6.5. A configuração ocupa três linhas.

```
spring.session.store-type=mongodb
spring.session.timeout=30m
spring.session.mongodb.collection-name=sessoes
```

O Spring Session cria automaticamente um índice de expiração sobre a coleção sessoes, e o próprio MongoDB remove os registros vencidos. Não é necessário escrever nenhuma rotina de limpeza.

Após a primeira execução, o banco apresenta duas coleções: usuarios, com as contas cadastradas, e sessoes, com os logins em andamento. Ambas podem ser inspecionadas pelo painel do Atlas, na opção Browse Collections.

5 CONTROLE DE ACESSO

5.1 Perfis de usuário

O sistema prevê três perfis, representados pelo enum Perfil. O quadro a seguir descreve o alcance de cada um.

Quadro 3 – Perfis de acesso do sistema

Perfil	Alcance
ADMIN	Acesso total, incluindo a gestão de contas e de perfis
GERENTE	Acesso ao painel e às páginas de relatórios
USUARIO	Acesso apenas ao próprio painel

Fonte: elaborado pelo autor (2026).

Toda conta criada pela tela pública de cadastro recebe o perfil USUARIO. A promoção para GERENTE ou ADMIN é feita exclusivamente por um administrador, pela tela de gestão de usuários.

5.2 Proteção das rotas

As regras de acesso estão reunidas na classe `SecurityConfig`. Cada padrão de URL é associado ao perfil necessário, e as regras são avaliadas na ordem em que foram declaradas.

```
.requestMatchers("/", "/login", "/cadastro",
    "/css/**", "/js/**", "/img/**").permitAll()
.requestMatchers("/admin/**").hasRole("ADMIN")
.requestMatchers("/gerencia/**").hasAnyRole("ADMIN", "GERENTE")
.anyRequest().authenticated()
```

A última regra, `anyRequest().authenticated()`, funciona como uma rede de proteção: qualquer rota criada no futuro fica automaticamente bloqueada para visitantes não autenticados, mesmo que o desenvolvedor esqueça de declará-la. A liberação passa a ser um ato deliberado, e não um descuido.

O método `hasRole` acrescenta internamente o prefixo `ROLE_` ao nome informado. Por esse motivo, o enum `Perfil` gera a autoridade no formato `ROLE_ADMIN`, enquanto a configuração menciona apenas `ADMIN`.

Nos templates, a mesma verificação é feita com o atributo `sec:authorize`, o que faz com que opções de menu inacessíveis sequer sejam exibidas ao usuário.

```
<a th:href="@{/admin/usuarios}"
    sec:authorize="hasRole('ADMIN')">Usuarios</a>
```

5.3 Criptografia das senhas

As senhas são gravadas como hash gerado pelo algoritmo `BCrypt`, configurado com fator de custo 12. O `BCrypt` incorpora um valor aleatório, chamado `salt`, a cada operação, de modo que duas contas com a mesma senha produzem hashes diferentes.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
```

O processo é irreversível: o sistema não consegue recuperar a senha original a partir do hash armazenado. Na autenticação, o `Spring Security` aplica o mesmo algoritmo à senha digitada e compara os resultados.

O fator de custo 12 determina o número de iterações executadas pelo algoritmo. Valores mais altos aumentam o tempo de verificação, dificultando ataques de força bruta, mas também tornam o login mais lento para o usuário legítimo.

5.4 Medidas complementares

Outras proteções foram aplicadas ao sistema:

- A proteção contra CSRF permanece ativa, e os formulários enviam o token automaticamente por meio do atributo `th:action` do Thymeleaf.
- O logout ocorre por meio de uma requisição POST, impedindo que a sessão seja encerrada por um simples link.
- O cookie de sessão é marcado como `HttpOnly`, o que impede sua leitura por código JavaScript.
- A mensagem de erro de login é genérica e não informa se o endereço de e-mail existe no sistema.
- Contas desativadas são rejeitadas na autenticação pelo método `isEnabled` da classe `UsuarioAutenticado`.
- Um administrador não consegue desativar nem excluir a própria conta, o que evita que o sistema fique sem nenhum administrador ativo.

6 PRINCIPAIS DECISÕES DE DESIGN

6.1 Separação entre tema e estrutura

O requisito de permitir a aplicação de temas visuais personalizados no futuro orientou a organização do CSS. Em vez de um único arquivo de estilos, o sistema utiliza dois: `base.css`, que define a estrutura e o posicionamento dos elementos, e um arquivo de tema, que contém apenas variáveis de cor, fonte e arredondamento.

```
:root {
  --fundo: #f4f6f8;
  --superficie: #ffffff;
  --texto: #1f2933;
  --destaque: #2563eb;
  --raio: 10px;
}
```

O arquivo `base.css` não contém nenhuma cor literal: todas as regras referenciam as variáveis. O tema carregado é definido por configuração, e o fragmento de cabeçalho monta o caminho do arquivo em tempo de execução.

```
<link rel="stylesheet" th:href="@{/css/base.css}">
<link rel="stylesheet" th:href="@{'/css/temas/' + ${tema.nome} +
'.css'}">
```

A variável `tema` é disponibilizada a todos os templates pela classe `TemaControllerAdvice`, anotada com `@ControllerAdvice`. Dessa forma, nenhum controller precisa incluí-la manualmente no modelo.

A consequência prática dessa decisão é que um novo tema não exige alterações em nenhum arquivo HTML ou Java: basta acrescentar um arquivo CSS e modificar uma linha de configuração.

6.2 Perfis como enum

Os perfis poderiam ter sido representados por textos livres gravados no banco. Optou-se pelo enum porque ele restringe os valores possíveis já no momento da compilação: um erro de digitação em um nome de perfil impede a construção do projeto, em vez de produzir uma falha silenciosa de autorização durante a execução.

Como contrapartida, a inclusão de um novo perfil exige alteração no código-fonte e nova implantação do sistema. Por se tratar de uma decisão estrutural, pouco frequente, essa limitação foi considerada aceitável.

6.3 Separação entre formulário e entidade

A tela de cadastro não manipula diretamente a entidade `Usuario`. Os dados digitados são recebidos pela classe `CadastroForm`, situada no pacote `dto`, que concentra as regras de validação.

```
@NotBlank(message = "Informe uma senha")
@Size(min = 8, max = 60, message = "A senha deve ter no minimo 8
caracteres")
private String senha;
```

Essa separação atende a duas finalidades. A primeira é evitar que campos sensíveis, como perfis e ativo, possam ser manipulados por meio de parâmetros

adicionais enviados no formulário. A segunda é permitir que o formulário tenha campos que não existem na entidade, como a confirmação de senha.

6.4 Criação automática do administrador

Um sistema recém-implantado apresenta um impasse: não é possível promover um usuário a administrador sem que já exista um administrador no banco. A solução adotada foi a classe `Cargainicial`, que implementa `CommandLineRunner` e é executada a cada inicialização.

A classe verifica a existência da conta administrativa e, caso ela não exista, cria-a com as credenciais definidas nas variáveis `ADMIN_EMAIL` e `ADMIN_SENHA`. Em execuções posteriores, nada é feito.

A senha definida em `ADMIN_SENHA` deve ser alterada antes de qualquer uso real do sistema, uma vez que o valor padrão está documentado publicamente neste texto e no arquivo de exemplo.

6.5 Sessões gravadas no banco

Por padrão, o Tomcat mantém as sessões na memória do servidor. Essa abordagem apresenta duas limitações: todos os usuários são desconectados a cada reinicialização da aplicação, e a solução não funciona quando há mais de uma instância do sistema em execução.

A adoção do Spring Session com MongoDB transfere as sessões para o banco de dados. Com isso, elas sobrevivem a reinicializações e podem ser compartilhadas entre várias instâncias. O requisito de armazenar as sessões no Atlas também é atendido por essa configuração.

6.6 Síntese das decisões

Quadro 4 – Síntese das decisões de design

Decisão	Motivo	Consequência
CSS dividido em base e tema	Permitir personalização visual futura	Tema novo não exige alteração de código
Perfis como enum	Impedir valores inválidos	Novo perfil exige nova implantação

Decisão	Motivo	Consequência
DTO separado da entidade	Proteger campos sensíveis	Mais uma classe por formulário
Administrador automático	Viabilizar o primeiro acesso	Senha padrão precisa ser trocada
Sessões no MongoDB	Sobreviver a reinicializações	Mais escritas no banco
Credenciais em variáveis	Manter segredos fora do repositório	Exige configuração antes de executar

Fonte: elaborado pelo autor (2026).

7 GUIA PARA FUTURAS ADAPTAÇÕES

Esta seção reúne roteiros para as modificações mais prováveis, indicando os arquivos envolvidos em cada caso.

7.1 Acrescentar um novo perfil

Para incluir um perfil denominado SUPERVISOR, por exemplo, são necessários três passos. O primeiro é acrescentar a constante ao enum Perfil:

```
SUPERVISOR("Supervisor"),
```

O segundo é declarar as regras de acesso correspondentes na classe SecurityConfig. O terceiro, caso o novo perfil deva enxergar itens específicos de menu, é ajustar os atributos sec:authorize no arquivo de layout. A tela de gestão de usuários passa a exibir o perfil automaticamente, pois ela percorre os valores do enum.

7.2 Criar um tema visual

A criação de um tema envolve apenas arquivos de estilo. Copia-se o arquivo padrao.css para um novo arquivo no mesmo diretório, alteram-se os valores das variáveis e informa-se o nome escolhido na configuração.

```
src/main/resources/static/css/temas/corporativo.css
app.tema.nome=corporativo
```

Caso o tema exija alterações estruturais, e não apenas de cores, novas variáveis podem ser criadas no arquivo de tema e referenciadas em base.css.

Recomenda-se manter todas as cores no arquivo de tema, preservando a separação entre elas.

7.3 Adicionar campos ao usuário

Para incluir um campo telefone, por exemplo, acrescenta-se o atributo e seus métodos de acesso na classe `Usuario`. Por ser o MongoDB um banco sem esquema fixo, não é necessária nenhuma migração: os documentos já existentes simplesmente não terão o novo campo, que será lido como nulo.

Se o campo precisar ser preenchido no cadastro, deve também ser declarado na classe `CadastroForm`, com as validações aplicáveis, copiado para a entidade no método `cadastrar` do `UsuarioService` e acrescentado ao formulário em `cadastro.html`.

7.4 Criar uma nova página protegida

Toda rota nova já nasce protegida pela regra `anyRequest().authenticated()`. Para restringi-la a um perfil específico, declara-se a regra correspondente em `SecurityConfig` antes dessa linha final.

Uma alternativa é anotar o método do controller com `@PreAuthorize`, o que exige acrescentar a anotação `@EnableMethodSecurity` à classe de configuração. A abordagem por URL foi mantida no projeto por concentrar todas as regras em um único arquivo, o que facilita a auditoria.

7.5 Substituir o banco de dados

A troca do MongoDB por um banco relacional afeta principalmente as camadas de modelo e repositório. Seria necessário substituir as anotações `@Document` e `@Id` pelas equivalentes da JPA, trocar `MongoRepository` por `JpaRepository` e transformar a lista de perfis em uma tabela de associação.

As camadas de serviço, controle e apresentação permaneceriam inalteradas, uma vez que não fazem referência direta à tecnologia de persistência. Essa independência é um dos benefícios da organização em camadas descrita na seção 3.1.

8 VERSIONAMENTO COM GITFLOW

O repositório segue o modelo GitFlow, que define branches com finalidades distintas. A branch main contém as versões estáveis, identificadas por tags. A branch develop reúne o desenvolvimento em andamento. Cada funcionalidade é desenvolvida em uma branch própria, criada a partir de develop.

Quadro 5 – Branches utilizadas no projeto

Branch	Conteúdo
main	Versão estável, identificada pela tag v1.0.0
develop	Integração das funcionalidades
feature/estrutura-inicial	Configuração do projeto Maven
feature/modelo-usuario	Entidade Usuario, enum Perfil e repositório
feature/spring-security	Autenticação, autorização e criptografia
feature/telas-thymeleaf	Telas de login, cadastro e painel
feature/administracao-usuarios	Tela de gestão de contas
feature/tema-configuravel	Separação entre base e tema
feature/documentacao	README e documentação técnica

Fonte: elaborado pelo autor (2026).

A integração das branches de funcionalidade em develop foi realizada com a opção --no-ff, que preserva no histórico o registro de que aquele conjunto de commits pertencia a uma funcionalidade específica. Para iniciar uma nova funcionalidade, o procedimento é o seguinte:

```
git checkout develop
git checkout -b feature/nome-da-funcionalidade

git checkout develop
git merge --no-ff feature/nome-da-funcionalidade
```

9 CONSIDERAÇÕES FINAIS

O sistema atende aos requisitos estabelecidos: cadastro, login e logout com tratamento adequado de senhas, controle de acesso por três perfis distintos, uso do Spring Security, armazenamento de usuários e sessões no MongoDB Atlas, interface construída em Thymeleaf com separação entre apresentação e lógica, e preparação para a aplicação de temas visuais personalizados.

A organização em camadas e a separação entre estrutura e tema foram as decisões que mais contribuíram para a facilidade de manutenção. Ambas permitem fazer alterações pontuais sem precisar compreender o sistema por completo.

Entre as evoluções possíveis, destacam-se a recuperação de senha por e-mail, o registro de auditoria das ações administrativas, a autenticação em duas etapas e a limitação do número de tentativas de login por endereço IP.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

MONGODB. MongoDB Atlas documentation. 2026. Disponível em: <https://www.mongodb.com/docs/atlas/>. Acesso em: 8 out. 2026.

PROVOS, Niels; MAZIÈRES, David. A future-adaptable password scheme. In: USENIX ANNUAL TECHNICAL CONFERENCE, 1999, Monterey. Proceedings. Berkeley: USENIX Association, 1999. p. 81-91.

SPRING. Spring Boot reference documentation. 2026. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: 8 out. 2026.

SPRING. Spring Security reference documentation. 2026. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 8 out. 2026.

THYMELEAF. Tutorial: using Thymeleaf. 2026. Disponível em: <https://www.thymeleaf.org/documentation.html>. Acesso em: 8 out. 2026.